Exercício Prático: Sistema de Ponto de Venda "Caixa Rápido"

Objetivo
O objetivo deste exercício é projetar e implementar um protótipo para o módulo de pagamentos de um sistema de Ponto de Venda (PDV), chamado "Caixa Rápido". Você explorará conceitos fundamentais da Programação Orientada a Objetos (POO) em Java, como:

- **Interfaces:** Definindo contratos para processadores de pagamento.
- **Polimorfismo:** Permitindo que diferentes métodos de pagamento sejam tratados de forma uniforme.
- **Encapsulamento:** Protegendo os dados dentro das classes.
- **Injeção de Dependência:** Passando o processador de pagamento para o carrinho de compras.
- **Métodos Default em Interfaces:** Fornecendo uma implementação padrão para emissão de recibos.

Contexto do Problema
Você foi contratado para desenvolver o núcleo de pagamentos de um novo sistema de PDV. O sistema deve ser flexível o suficiente para aceitar múltiplos métodos de pagamento (Pix, Cartão de Crédito e Boleto) sem a necessidade de alterar o código que gerencia o carrinho de compras ou calcula o total da venda. A escolha do método de pagamento deve ocorrer em tempo de execução, com base na seleção do usuário.

Requisitos do Sistema
O sistema será composto por uma interface e diversas classes. Siga as especificações detalhadas abaixo para a implementação:

1. Interface: ProcessadorPagamento
Esta interface define o contrato que todos os métodos de pagamento devem seguir.

- **Métodos Absratos:**
  - boolean pagar(double valor): Este método deve conter a lógica simulada para processar o pagamento. Ele retorna true se o pagamento for bem-sucedido e false caso contrário.

- **Método Default (Padrão):**
  - default void emitirRecibo(double valor): Implementa uma lógica padrão para exibir uma mensagem de confirmação de pagamento no console, formatando o valor como moeda (ex: R$ 10,00). Esta mensagem deve ser impressa apenas se o método pagar retornar true.

2. Implementações de ProcessadorPagamento
Você deve criar três classes que implementam a interface ProcessadorPagamento. Cada uma simulará o processamento de uma forma diferente.

- **Classe PagamentoPix**
  - **Atributos:** chavePix (String).
  - **Construtor:** Recebe e inicializa a chavePix.
  - **Lógica do método pagar:**
    - Verifica se o valor é maior que zero e se a chavePix não é nula ou vazia.
    - Simula o processamento exibindo mensagens no console ("Processando PIX...", "Pagamento aprovado...").
    - Retorna true se as validações passarem, caso contrário, imprime mensagem de erro e retorna false.

- **Classe PagamentoCartaoCredito**
    - **Atributos:** numero (String), nomeTitular (String), cvv (String), e um atributo privado limiteSimulado (double) inicializado com um valor fixo (ex: 5000.0).
    - **Construtor:** Recebe e inicializa numero, nomeTitular e cvv.
    - **Lógica do método pagar:**
        - Verifica se todos os campos (numero, nome, cvv) não são nulos ou vazios e se o valor da compra é menor ou igual ao limiteSimulado.
        - Simula a autorização exibindo mensagens no console incluindo o nome do titular.
        - Retorna true se as validações passarem, caso contrário, imprime mensagem de erro ("Pagamento no cartão recusado") e retorna false.

- **Classe PagamentoBoleto**
    - **Lógica do método pagar:**
        - Verifica se o valor é maior que zero.
        - Gera uma "linha digitável" simulada (pode usar UUID.randomUUID().toString()).
        - Imprime a linha digitável e instruções de pagamento no console.
        - Retorna true se o valor for válido, caso contrário, imprime erro e retorna false.

3. Classe de Domínio: Item
Representa um produto no carrinho.
- **Atributos:** descricao (String), preco (double).
- **Construtor:** Inicializa os atributos.
- **Métodos:** Getters para os atributos.

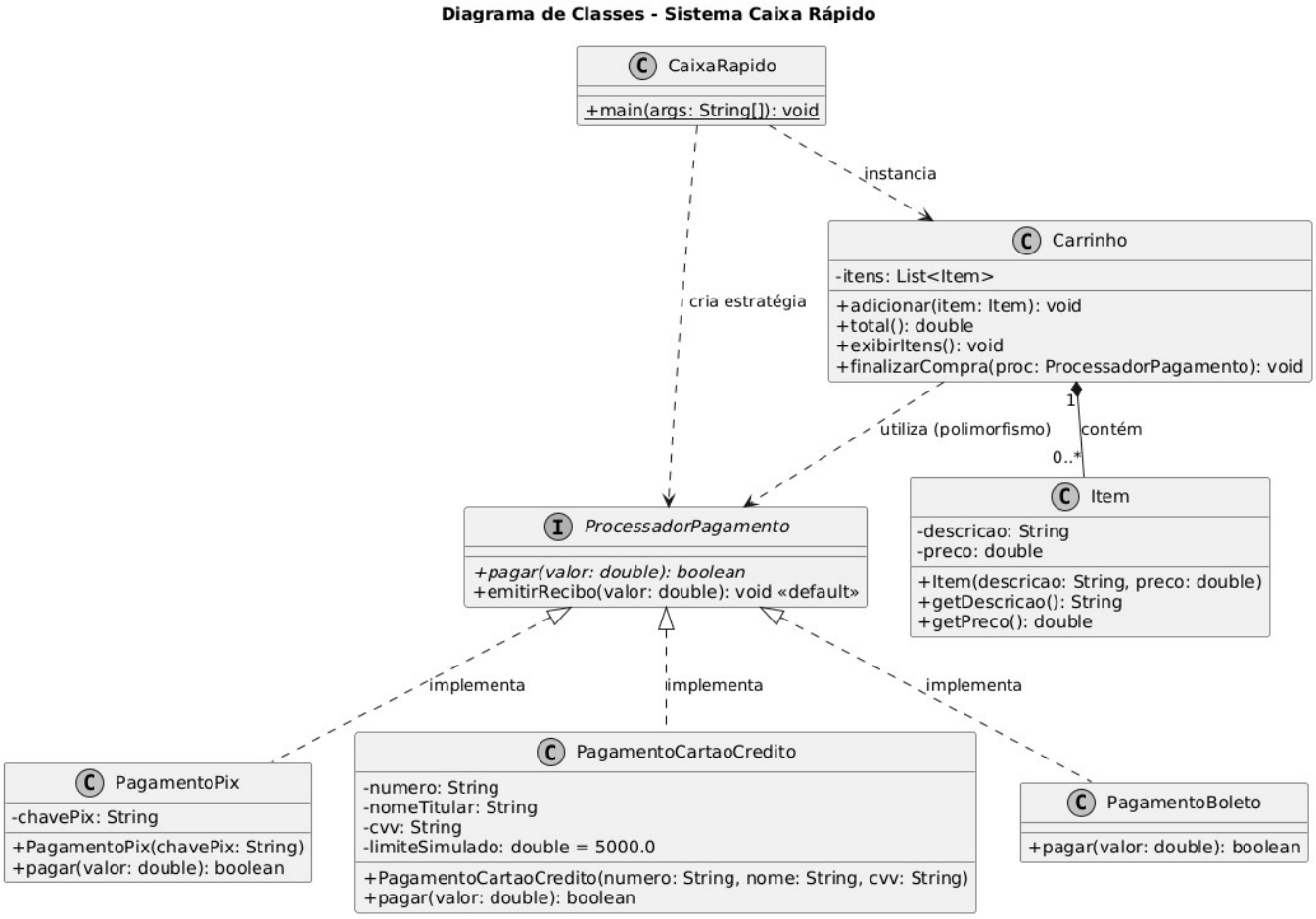
4. Classe de Negócio: Carrinho
Gerencia os itens da compra e a finalização do pagamento.
- **Atributos:** Uma lista privada de itens (List<Item> itens), inicializada como um ArrayList.
- **Métodos:**
    - void adicionar(Item item): Adiciona um item à lista.
    - double total(): Calcula e retorna a soma dos preços de todos os itens no carrinho.
    - void exibirItens(): Imprime no console a lista de itens, seus preços formatados e o valor total do carrinho.
    - void finalizarCompra(ProcessadorPagamento proc): Este é o ponto crucial do polimorfismo. O método recebe *qualquer* implementação da interface ProcessadorPagamento.
        - Calcula o total do carrinho chamando total().
        - Chama o método pagar(valor) do processador recebido.
        - Se o pagamento for bem-sucedido (true), chama o método emitirRecibo(valor) do processador.
        - Se falhar (false), imprime uma mensagem de erro ("Pagamento recusado...").

5. Classe Principal: CaixaRapido
Contém o método main e serve como ponto de entrada da aplicação.
- Cria uma instância de Carrinho.
- Adiciona 3 itens de exemplo ao carrinho (ex: Café, Pão de Queijo, Suco com seus respectivos preços).
- Chama carrinho.exibirItens() para mostrar o resumo da compra.
- Solicita que o usuário escolha o método de pagamento através do console (1-Pix, 2-Cartão, 3-Boleto) usando a classe Scanner.
- Utiliza uma estrutura switch para:
    - Solicitar os dados específicos do método escolhido (ex: chave Pix para Pix, dados do cartão para Cartão).
    - Instanciar a classe de pagamento correspondente (ex: new PagamentoPix(...)).
    - Atribuir a instância criada a uma variável do tipo da interface ProcessadorPagamento.
- Chama carrinho.finalizarCompra(processador), passando a instância criada.
- Fecha o Scanner.

Exemplo de Saída Esperada (Console)

```
Plaintext
Carrinho:
- Café: R$ 8,50
- Pão de Queijo: R$ 6,00
- Suco: R$ 9,90
Total: R$ 24,40

Selecione o meio de pagamento [1-Pix, 2-Cartão, 3-Boleto]: 1
Digite a chave Pix: aluno@exemplo.com
Processando PIX para chave: aluno@exemplo.com ...
Pagamento aprovado via PIX!
Recibo: pagamento de R$ 24,40 confirmado.
```

Diagrama de Classes UML

Abaixo está o diagrama de classes que representa a estrutura do sistema "Caixa Rápido".